This project is best treated as an **industry-level capstone project** rather than a simple chatbot. The guide below is designed so that students can complete it, while learning modern AI concepts such as Multi-Agent Systems, Retrieval-Augmented Generation (RAG), vector databases, and LLM integration.

Project Title

Multi-Agent AI Customer Support Assistant using RAG and LLMs

---

1. Project Objective

Design and develop a web-based AI-powered customer support assistant capable of answering customer queries using multiple specialized AI agents.

Unlike a normal chatbot, this system should:

- Understand customer intent
- Route the request to the correct AI agent
- Retrieve relevant company information
- Generate accurate responses
- Maintain conversation history
- Escalate unresolved issues

---

2. Learning Outcomes

After completing this project, students should understand:

- Multi-Agent Systems
- Large Language Models (LLMs)
- Retrieval-Augmented Generation (RAG)
- Embedding Models
- Vector Databases
- REST APIs
- Full Stack Development
- Cloud Deployment

---

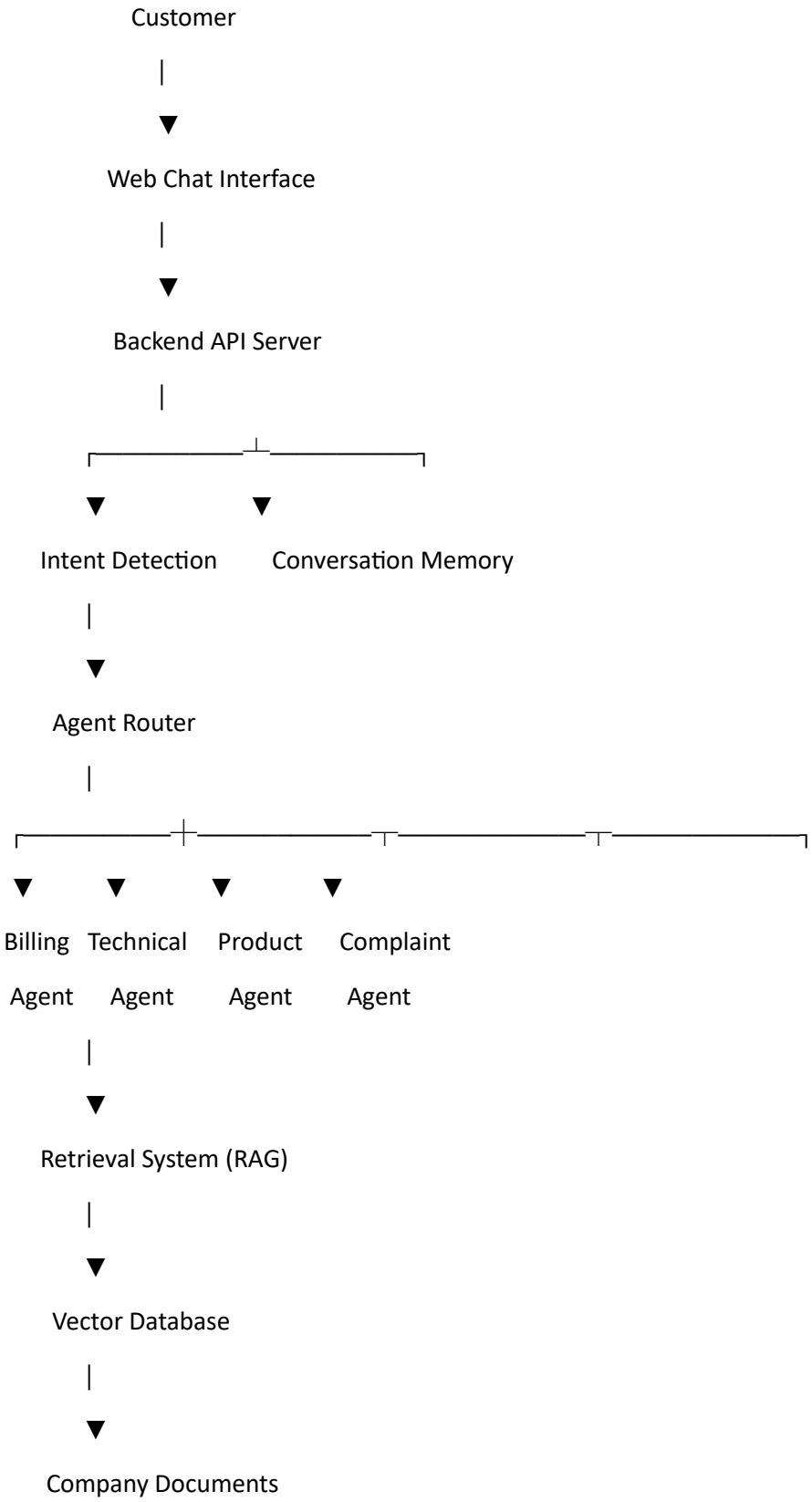
3. Problem Statement

Companies receive thousands of customer queries every day. A single chatbot often struggles to answer questions from different domains such as billing, technical support, product information, and complaints.

The objective is to build a **Multi-Agent AI System** where each AI agent specializes in one domain, and a central orchestrator routes queries to the appropriate agent(s).

---

4. Overall System Architecture



|

▼

Response Aggregator

|

▼

Final Response

---

5. Technology Stack

Frontend

- React.js
- Next.js
- Tailwind CSS
- Axios

Backend

- Python FastAPI (Recommended)
- OR Node.js + Express

AI Models

Students may use:

- OpenAI GPT
- Google Gemini
- Llama 3 (via Ollama or Groq)
- Hugging Face models

Embedding Models

Recommended:

- sentence-transformers/all-MiniLM-L6-v2
- BAAI/bge-small-en-v1.5

Vector Database

Choose one:

- FAISS (Recommended)
- ChromaDB
- Pinecone (Cloud)

Database

- MongoDB
- PostgreSQL

Deployment

Frontend

- Vercel

Backend

- Railway
- Render

Database

- MongoDB Atlas

---

6. Software Requirements

- Python 3.11+
- Node.js 20+
- VS Code
- Git
- Docker (Optional)

Python Libraries:

- FastAPI
- LangChain
- LangGraph (Optional)
- FAISS
- ChromaDB
- Sentence Transformers

- OpenAI SDK
- PyPDF
- Pandas
- Uvicorn

---

7. Functional Modules

Module 1

User Authentication

Features

- Login
- Register
- Session management

Module 2

Chat Interface

Features

- Chat window
- Send message
- Conversation history
- Typing indicator

Module 3

Intent Detection Agent

Responsibilities

Determine whether the customer query belongs to:

- Billing
- Refund
- Product
- Technical Support
- Complaint
- General FAQ

Module 4

Agent Router

Routes requests to one or multiple specialized agents.

Example:

Query:

"I paid yesterday but Premium is still locked."

Should invoke

- Billing Agent
- Technical Agent

Module 5

Specialized Agents

Billing Agent

Handles

- Payment issues
- Subscription
- Invoice
- Refund

Technical Support Agent

Handles

- Login
- Password reset
- Installation
- Errors
- Bugs

Product Agent

Handles

- Features

- Pricing
- Comparisons
- Availability

Complaint Agent

Handles

- Complaints
- Escalation
- Customer dissatisfaction

FAQ Agent

Handles

- Company policies
- General questions
- Contact information

Module 6

Knowledge Base

Store documents such as

- FAQ.pdf
- UserManual.pdf
- RefundPolicy.pdf
- Warranty.pdf
- ShippingPolicy.pdf
- Pricing.pdf

Module 7

Retrieval-Augmented Generation (RAG)

Workflow

Documents

↓

Split into chunks

↓

Generate embeddings

↓

Store embeddings

↓

User asks question

↓

Retrieve relevant chunks

↓

Pass retrieved context to LLM

↓

Generate answer

Module 8

Conversation Memory

Store

- User message
- AI response
- Timestamp
- Session ID

Module 9

Analytics Dashboard (Optional)

Display

- Number of conversations
- Agent usage
- Response time
- Satisfaction score

---

8. Folder Structure

```
customer-support-ai/
|
├── frontend/
|   ├── components/
|   ├── pages/
|   ├── hooks/
|   ├── services/
|   └── styles/
|
├── backend/
|   ├── api/
|   ├── agents/
|   |   billing.py
|   |   technical.py
|   |   product.py
|   |   complaint.py
|   |   faq.py
|   |   router.py
|   |
|   ├── rag/
|   ├── embeddings/
|   ├── vectorstore/
|   ├── database/
|   ├── models/
|   └── main.py
|
├── knowledge_base/
|    faq.pdf
|    refund_policy.pdf
|    shipping_policy.pdf
```

warranty.pdf

user_manual.pdf

datasets/

README.md

requirements.txt

---

9. Working Public Datasets

Students are encouraged to use **both public datasets and a custom company knowledge base**.

A. Consumer Complaint Dataset (Official CFPB)

Contains millions of real customer complaints, issue categories, company responses, and narratives. Updated daily. (Consumer Financial Protection Bureau)

Official download page:

Consumer Complaint Database (CFPB)

B. Banking77 Intent Classification Dataset

Excellent for intent detection with 77 banking-related customer intents.

Official dataset:

Banking77 Dataset on Hugging Face

C. DailyDialog Dataset

High-quality multi-turn conversations useful for dialogue modeling.

Official dataset:

https://github.com/liuzeming01/XDailyDialog

D. SQuAD Dataset

Question-answering dataset useful for retrieval and answer generation.

Official dataset:

https://github.com/rajpurkar/SQuAD-explorer?utm_source=chatgpt.com

The dataset files are directly available in the repository:

- train-v2.0.json

- dev-v2.0.json

E. MS MARCO Dataset

Large-scale question-answer dataset suitable for semantic retrieval systems. (arXiv)

Official dataset:

https://github.com/microsoft/MSMARCO-Question-Answering?utm_source=chatgpt.com

---

10. Creating the Company Knowledge Base

Students should create a fictional company (e.g., **TechMart Electronics**) and prepare:

knowledge_base/

FAQ.pdf

RefundPolicy.pdf

ShippingPolicy.pdf

Warranty.pdf

Pricing.pdf

Products.pdf

InstallationGuide.pdf

UserManual.pdf

This knowledge base is ingested into the vector database and forms the basis for RAG.

---

11. Step-by-Step Implementation Plan

1 – Planning

- Gather requirements
- Design system architecture
- Create UI wireframes
- Set up Git repository

2 – Frontend

- Build login page
- Create chat interface
- Implement conversation history
- Connect frontend to backend

3 – Backend

- Develop REST APIs
- Implement authentication
- Set up MongoDB
- Create session management

4 – AI Agent Development

- Implement Intent Detection Agent
- Develop Billing, Technical, Product, Complaint, and FAQ Agents
- Build Agent Router

5 – RAG Pipeline

- Prepare company documents
- Chunk text
- Generate embeddings
- Store embeddings in FAISS or ChromaDB
- Implement semantic retrieval

6 – LLM Integration

- Integrate OpenAI, Gemini, or Llama
- Create prompts for each agent
- Combine retrieved context with user queries

7 – Testing & Evaluation

- Test agent routing
- Evaluate retrieval quality
- Measure response time
- Test edge cases and error handling

8 – Deployment

- Deploy frontend to Vercel
- Deploy backend to Railway or Render
- Connect MongoDB Atlas
- Conduct end-to-end testing

---

12. Evaluation Criteria

Component	Marks
Frontend Design	10
Backend APIs	15
Multi-Agent Architecture	20
RAG Implementation	20
LLM Integration	15
Database Design	10
Documentation & Deployment	10
Total	**100**

---

13. Project Deliverables

Students should submit:

1. Source code (frontend and backend)
2. Project report (PDF)
3. README with setup instructions
4. Demonstration video (compulsory)
5. Knowledge base documents (PDFs)
6. Sample datasets (if used)
7. Deployment links (if deployed)

---

14. Suggested Enhancements (Bonus)

- Voice-enabled customer support
- Multilingual conversations
- Sentiment analysis for routing frustrated customers
- Automatic ticket creation
- Human-agent handoff
- Email and WhatsApp integration
- AI-generated conversation summaries
- Admin dashboard to update the knowledge base
- Customer satisfaction feedback and analytics

This guide follows a modern enterprise architecture, emphasizing **specialized AI agents coordinated through an orchestrator, backed by Retrieval-Augmented Generation (RAG)**. It exposes students to practical software engineering patterns, scalable AI design, and cloud deployment practices that are widely used in production AI customer support systems today.